# ORACLE GOLDENGATE

## From Beginner to Real-Time Production Expert

A Senior Oracle DBA's Complete Production Handbook

Real Commands. Real Scenarios. Real Production Insights.

**For Oracle DBAs Targeting L2/L3 Interviews**

Covering GoldenGate 19c / 21c / 23ai

# Chapter 1: Introduction to Oracle GoldenGate

## 1.1 What is Oracle GoldenGate?

Oracle GoldenGate is an enterprise-grade, real-time data integration and replication platform that operates at the transaction log level. It captures committed changes directly from database redo/archive logs and delivers them to one or more target systems with near-real-time latency — typically seconds, and sub-second in well-optimized environments. Unlike traditional ETL tools that run in batch windows, GoldenGate moves data continuously, making it the backbone of any always-on, mission-critical architecture. It is not just a replication tool; it is a change data capture (CDC) engine that underpins zero-downtime migrations, active-active databases, and real-time analytics.

**GoldenGate does NOT use database links or SQL-level replication. It reads directly from redo logs, resulting in minimal to moderate overhead on the source depending on workload volume and configuration — typically well within acceptable limits on a properly tuned production system.**

## 1.2 Why Companies Use GoldenGate — Real Production Use Cases

### Use Case 1: Zero Downtime Database Migration (Banking)

In real-world banking migrations from Oracle 12c on-premise to Oracle 19c on OCI (Oracle Cloud Infrastructure), downtime is simply not acceptable — ATMs, internet banking, and payment gateways operate 24/7. GoldenGate bidirectional sync between source and target during migration enables production cutover windows as short as a few minutes — essentially a connection string switch. Without GoldenGate, such migrations typically require 6 to 8 hours of scheduled downtime.

### Use Case 2: Real-Time Reporting Without OLTP Impact (Retail Banking)

Core banking runs on OLTP-optimized schemas. When BI teams run heavy analytical queries on the same database, the result is lock contention, CPU spikes, and I/O saturation. The solution: unidirectional GoldenGate replication to a dedicated reporting replica, with column transformations applied in-flight via GoldenGate's COLMAP. Reports run on the replica with zero impact on production OLTP.

### Use Case 3: Active-Active for Disaster Recovery (Telecom)

Telecom environments often require an RPO of zero and RTO under 30 seconds. Oracle Data Guard provides a standby, but Active Data Guard supports only real-time read queries — not writes. GoldenGate active-active configuration provides two fully writable data centres with conflict detection and resolution logic (CDR), enabling near-zero-RPO failover without application-level changes.

### Use Case 4: Microservices Data Mesh (Fintech)

In modern fintech architectures, multiple microservices own their own PostgreSQL, MySQL, or Oracle schemas. GoldenGate serves as the CDC event backbone — changes in a loan origination Oracle

DB are replicated to Kafka topics via GoldenGate for Big Data, consumed by downstream services in real time. This is the foundation of a CDC-driven event-driven architecture.

## 1.3 GoldenGate vs. Oracle Data Guard — The Real Comparison

This is one of the most common architecture questions in GoldenGate interviews. Here is an honest, production-focused comparison:

| Feature | GoldenGate | Data Guard |
|---|---|---|
| Primary Purpose | Heterogeneous replication / CDC / ETL | Disaster Recovery / High Availability |
| Target DB Type | Any DB: Oracle, MySQL, PostgreSQL, Kafka... | Oracle only |
| Standby Writeable? | Yes (Active-Active fully supported) | Active Data Guard supports real-time read queries; not full writes |
| Replication Granularity | Table-level, row-level, column-level filters | Entire database (block-level) |
| Network Protocol | TCP/IP (custom) | Oracle Net / Redo shipping |
| Latency | Near real-time (seconds; sub-second in optimized environments) | Near-zero (synchronous mode) |
| Transformations | Yes — in-flight column mapping, filters, SQL | No — block-for-block copy |
| Multi-Source/Target | Many-to-many topologies supported | 1 primary to multiple standbys |
| License Cost | Additional license required | Included with Enterprise Edition |
| Best For | Migration, reporting, active-active, ETL | Pure HA/DR scenarios |

**Production rule of thumb: Use Data Guard for same-version, same-platform Oracle HA/DR. Use GoldenGate when heterogeneous replication, data transformation, active-active writes, or granular table-level control is required. The two are complementary — many enterprises run both.**

# Chapter 2: Architecture Deep Dive

Understanding GoldenGate architecture is not about memorising terms — it is about visualising data flow to diagnose latency, plan capacity, and design topologies correctly. Each component has a specific responsibility, and a clear mental model of how they interact is essential for both production work and interview success.

## 2.1 The Three-Process Architecture

GoldenGate's core architecture revolves around three logical process types: Extract, Data Pump, and Replicat. Each has a specific role, and trail files act as the durable message queue between them.

### EXTRACT — The Change Capture Engine

Extract is the source-side process. It attaches to the database redo log stream (in Integrated mode) or reads archive logs directly (Classic mode) and captures every committed DML (INSERT, UPDATE, DELETE) and DDL change for the configured tables.

Extract writes captured changes to a LOCAL TRAIL FILE on the source server. Trail files are binary, sequenced files (e.g., ./dirdat/lt000001, lt000002...) that act as GoldenGate's internal journal.

- Integrated Extract: Uses the Oracle LogMiner infrastructure via a server-side capture process. Oracle-recommended for 11.2+. Internally uses STREAMS_POOL memory allocation. Supports compressed tables, IOTs, encrypted columns, and LOBs natively.
- Classic Extract: Reads redo logs directly using GoldenGate's own log parser. Required for Oracle versions below 11.2 or specific scenarios where LogMiner infrastructure is unavailable.

### DATA PUMP — The Network Transport Agent

Data Pump is a secondary Extract process running on the source server. It reads the LOCAL TRAIL file written by the primary Extract and transmits the data over TCP/IP to the TARGET SERVER, writing to a REMOTE TRAIL FILE there.

The reason for separating Extract and Pump is resilience. If the network between source and target goes down, Extract continues writing to the local trail up to the configured trail retention limit. When the network recovers, Pump resumes from its checkpoint and delivers everything without data loss. Without Pump, the primary Extract would be tightly coupled to network availability.

> **In production, always run Extract and Data Pump as separate processes. A single Extract writing directly to the remote trail creates tight network coupling — any network interruption will stall capture, not just delivery.**

### REPLICAT — The Target Apply Engine

Replicat runs on the TARGET server. It reads the remote trail files and applies changes to the target database using standard DML. Three modes are available in modern GoldenGate:

- Classic Replicat: Single-threaded, serial apply. Simple to configure but lowest throughput.

- Integrated Replicat (IR): Uses the database's apply infrastructure. Supports parallelism with dependency-aware transaction ordering.
- Parallel Replicat (PR): Introduced in GoldenGate 18c. Multiple apply threads with automatic workload distribution. The recommended choice for high-throughput targets.

## 2.2 Trail Files — GoldenGate's Durable Message Queue

Trail files are the foundation of GoldenGate's reliability model. They are sequential binary files written to disk, acting as a persistent buffer between processes.

- Default location: $OGG_HOME/dirdat/
- Naming convention: 2-character prefix + 6-digit sequence number (e.g., lt000001, rt000001)
- Default size: 500 MB per file before GoldenGate rotates to the next sequence
- Never delete trail files manually — always use PURGEOLDEXTRACTS in the Manager parameter file
- Trail files persist on disk until all downstream processes (Pump or Replicat) have consumed them past their checkpoint

**⚠ A full filesystem under $OGG_HOME/dirdat will immediately abend ALL GoldenGate processes. This is the single most common cause of production GoldenGate outages. Always mount dirdat on a dedicated volume with filesystem monitoring and alerting. A stuck Replicat can accumulate days of trail files very quickly on high-transaction-rate systems.**

## 2.3 Checkpoints — GoldenGate's Recovery Mechanism

Checkpoints are persistent position markers recorded by each GoldenGate process to enable crash-safe recovery. They are stored in the $OGG_HOME/dirchk/ directory.

- Extract checkpoint: Position in the source redo log expressed as log sequence number + RBA (Relative Byte Address)
- Pump checkpoint: Current read position in the local trail file
- Replicat checkpoint: Current read position in the remote trail file

GoldenGate supports two types of Replicat checkpoints: file-based checkpoints stored in dirchk/, and database checkpoint tables (CHECKPOINTTABLE). The database checkpoint table is more reliable because it is committed as part of the same transaction as the applied data, providing an atomically consistent recovery point. File-based checkpoints are updated separately and may be slightly behind in a crash scenario. Production environments should always use a CHECKPOINTTABLE.

Because of checkpoints, GoldenGate is crash-safe. When a Replicat is killed mid-transaction, it restarts from its last committed checkpoint and re-applies any un-committed work. HANDLECOLLISIONS handles any duplicate-key situations during this recovery phase.

## 2.4 The Manager Process

Manager is the GoldenGate watchdog process. It must be running before any other GoldenGate process can start. Its responsibilities include:

- Port management (default port 7809 for Classic architecture; 9100+ for Microservices Architecture)

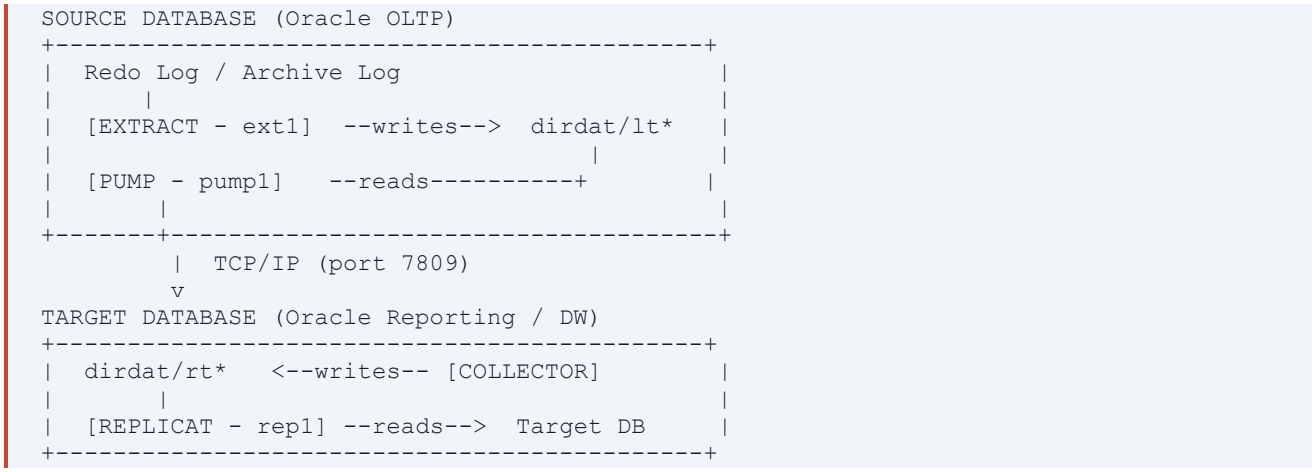
- Process lifecycle management — auto-restart of abended processes based on AUTORESTART configuration
- Trail file cleanup coordination via PURGEOLDEXTRACTS
- Dynamic port allocation for Collector processes used by Data Pump

**Manager parameter file — production-ready sample:**

```
-- $OGG_HOME/dirprm/mgr.prm
PORT 7809
DYNAMICPORTLIST 7810-7820
PURGEOLDEXTRACTS ./dirdat/*, USECHECKPOINTS, MINKEEPDAYS 3
AUTORESTART EXTRACT *, WAITMINUTES 2, RESETMINUTES 60
AUTORESTART REPLICAT *, WAITMINUTES 2, RESETMINUTES 60
LAGREPORTHOURS 1
LAGREPORTMINUTES 30
```

## 2.5 Architecture Diagram (Text Representation)

The following diagram represents a complete unidirectional GoldenGate pipeline. Understanding this flow is essential for diagnosing latency and answering architecture questions:



# Chapter 3: Installation & Setup (Step-by-Step)

Before touching the GoldenGate installer, the source database must be correctly prepared. Skipping these prerequisites is the most common reason for mysterious Extract abends days after a seemingly successful setup.

## 3.1 Prerequisites — Database Configuration

### Step 1: Enable ARCHIVELOG Mode

```
-- Check current mode
SQL> SELECT LOG_MODE FROM V$DATABASE;

-- Enable ARCHIVELOG (requires a controlled restart)
SQL> SHUTDOWN IMMEDIATE;
SQL> STARTUP MOUNT;
SQL> ALTER DATABASE ARCHIVELOG;
SQL> ALTER DATABASE OPEN;
SQL> ARCHIVE LOG LIST;
```

### Step 2: Enable FORCE LOGGING

FORCE LOGGING ensures that even NOLOGGING operations are written to the redo stream. Without this, Direct Path Inserts and CREATE TABLE AS SELECT operations will be silently missed by GoldenGate Extract.

```
SQL> ALTER DATABASE FORCE LOGGING;
SQL> SELECT FORCE_LOGGING FROM V$DATABASE; -- Expected: YES
```

### Step 3: Enable Supplemental Logging

Supplemental logging adds additional column data to redo log records so GoldenGate can identify precisely which rows changed and reconstruct full before/after images. Minimum-level supplemental logging is not sufficient for GoldenGate — ALL COLUMNS logging is required for UPDATE operations to work correctly.

```
-- Minimal (required at the database level)
SQL> ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;

-- All columns (recommended for GoldenGate source databases)
SQL> ALTER DATABASE ADD SUPPLEMENTAL LOG DATA ALL COLUMNS;

-- Or at the table level for granular control
SQL> ALTER TABLE hr.employees ADD SUPPLEMENTAL LOG DATA ALL COLUMNS;

-- Verify
SQL> SELECT SUPPLEMENTAL_LOG_DATA_MIN, SUPPLEMENTAL_LOG_DATA_ALL
     FROM V$DATABASE;
```

### Step 4: Enable GoldenGate Replication (for Integrated Extract)

```
SQL> ALTER SYSTEM SET enable_goldengate_replication=TRUE SCOPE=BOTH;
```

### Step 5: Create the GoldenGate Administration User

```
-- CDB-level common user (for Oracle 12c+ Multitenant)
SQL> CREATE USER c##ggadmin IDENTIFIED BY Welcome#123
     DEFAULT TABLESPACE USERS
     TEMPORARY TABLESPACE TEMP
     QUOTA UNLIMITED ON USERS;

SQL> GRANT DBA TO c##ggadmin CONTAINER=ALL;

SQL> EXEC DBMS_GOLDENGATE_AUTH.GRANT_ADMIN_PRIVILEGE(
       grantee             => 'c##ggadmin',
       privilege_type      => 'CAPTURE',
       grant_select_privileges => TRUE,
       container           => 'ALL');
```

## 3.2 GoldenGate Software Installation on Linux

### Step 1: Prepare Directories and Extract Installer

```
mkdir -p /u01/app/oracle/product/goldengate/19c
cd /tmp
unzip V982764-01.zip
unzip fbo_ggs_Linux_x64_Oracle_shiphome.zip -d /tmp/gg_install
```

### Step 2: Run the Oracle Universal Installer

```
cd /tmp/gg_install/fbo_ggs_Linux_x64_Oracle_shiphome/Disk1
./runInstaller

# For automated/CI deployments, use silent install:
./runInstaller -silent -responseFile /tmp/gg_install.rsp
```

**Silent response file (gg_install.rsp):**

```
oracle.install.responseFileVersion=/oracle/install/rspfmt_ogginstall_response_schema_v19_
1_0
INSTALL_OPTION=ORA19c
SOFTWARE_LOCATION=/u01/app/oracle/product/goldengate/19c
START_MANAGER=false
MANAGER_PORT=7809
DATABASE_LOCATION=/u01/app/oracle/product/db_home/19c
INVENTORY_LOCATION=/u01/app/oraInventory
UNIX_GROUP_NAME=oinstall
```

### Step 3: Set Environment Variables

```
export OGG_HOME=/u01/app/oracle/product/goldengate/19c
export LD_LIBRARY_PATH=$OGG_HOME:$LD_LIBRARY_PATH
export PATH=$OGG_HOME:$PATH
export ORACLE_SID=PRODDB
export ORACLE_HOME=/u01/app/oracle/product/db_home/19c

# Persist to shell profile
echo 'export OGG_HOME=/u01/app/oracle/product/goldengate/19c' >> ~/.bash_profile
echo 'export PATH=$OGG_HOME:$PATH' >> ~/.bash_profile
```

### Step 4: Create GoldenGate Working Subdirectories

```
cd $OGG_HOME && ./ggsci
GGSCI> CREATE SUBDIRS

# Creates the following structure:
# dirprm/  -- parameter files for all processes
# dirdat/  -- trail files (local and remote)
# dirchk/  -- checkpoint files
# dirrpt/  -- process report files
# dirpcs/  -- process status files
# dirtmp/  -- temporary working files
```

## Step 5: Configure Database Credentials (Credential Store)

```
# Direct login (testing only -- never use plain-text passwords in production param files)
GGSCI> DBLOGIN USERID c##ggadmin, PASSWORD Welcome#123

# Production: use the encrypted credential store
GGSCI> ADD CREDENTIALSTORE
GGSCI> ALTER CREDENTIALSTORE ADD USER c##ggadmin, PASSWORD Welcome#123, ALIAS ggalias
GGSCI> DBLOGIN USERIDALIAS ggalias
```

# Chapter 4: Real-Time Scenarios

## 4.1 Unidirectional Replication — OLTP to Reporting

The most common GoldenGate topology. The OLTP database (HR, Finance, Core Banking) is the source, and a reporting or analytics database is the target. Changes flow in one direction only: source to target.

Business driver: Analytical queries are long-running, CPU-intensive, and cause lock contention on OLTP schemas. A dedicated reporting replica eliminates this conflict entirely.

### Design Considerations for This Topology

- Use Integrated Extract on the source (Oracle 11.2 and above)
- Use Parallel Replicat on the target for throughput
- Apply DDL replication selectively — never replicate DROP or TRUNCATE without a change control gate
- Consider column transformations if the reporting schema is denormalized
- Set FETCHOPTIONS FETCHPKUPDATECOLS on Extract to ensure the full row is fetched on primary key updates

### Parameter File — Extract (ext_rpt)

```
-- dirprm/ext_rpt.prm
EXTRACT ext_rpt
USERIDALIAS ggalias DOMAIN OracleGoldenGate
EXTTRAIL ./dirdat/lt

-- Capture only the tables required for reporting
TABLE hr.employees;
TABLE hr.departments;
TABLE fin.accounts;
TABLE fin.transactions;

-- Selective DDL -- structural ALTERs only, never blanket DDL ALL
DDL INCLUDE MAPPED OBJNAME hr.*, OBJNAME fin.*
DDLOPTIONS REPORT
```

### Parameter File — Data Pump (pump_rpt)

```
-- dirprm/pump_rpt.prm
EXTRACT pump_rpt
USERIDALIAS ggalias DOMAIN OracleGoldenGate
RMTHOST target-db-server.internal, MGRPORT 7809
RMTTRAIL ./dirdat/rt
PASSTHRU
TABLE hr.*;
TABLE fin.*;
```

### Parameter File — Replicat (rep_rpt)

```
-- dirprm/rep_rpt.prm
```

```
REPLICAT rep_rpt
USERIDALIAS ggalias DOMAIN OracleGoldenGate
ASSUMETARGETDEFS

-- Map source tables to the reporting schema
MAP hr.employees,    TARGET hr_rpt.employees;
MAP hr.departments,  TARGET hr_rpt.departments;
MAP fin.accounts,    TARGET fin_rpt.accounts;
MAP fin.transactions,TARGET fin_rpt.transactions;
```

## 4.2 Bidirectional Replication — Active-Active

The most architecturally complex GoldenGate topology. Both databases are fully writable and changes flow in both directions. The core challenge: what happens when the same row is updated on both sides simultaneously? Conflict Detection and Resolution (CDR) is critical.

### Conflict Detection and Resolution Strategy

In active-active deployments, partitioning writes by geography or business unit is the recommended approach to minimise conflict probability. For example, DC-Mumbai handles write traffic from one region, DC-Hyderabad from another. Conflicts are rare by design but must still be handled explicitly:

```
-- Replicat parameter file with CDR
REPLICAT rep_hy_to_mum
USERIDALIAS ggalias

-- Timestamp-based LAST WRITER WINS conflict resolution
MAP fin.accounts, TARGET fin.accounts,
  RESOLVECONFLICT (UPDATEROWEXISTS,
    (DEFAULT, USEMAX (last_update_ts)),
    (DEFAULT, OVERWRITE)),
  RESOLVECONFLICT (INSERTROWEXISTS,
    (DEFAULT, OVERWRITE));
```

### Loop Prevention — Critical for Active-Active

Without loop prevention, a change applied by Replicat on DC-A would be captured by Extract on DC-A, sent to DC-B, applied there, captured again, and looped back indefinitely. Transaction tagging prevents this:

```
-- Extract on DC-A: ignore transactions tagged by Replicat
EXTRACT ext_a
TRANLOGOPTIONS EXCLUDETAG 11

-- Replicat on DC-A: tag every transaction it applies with value 11
REPLICAT rep_b_to_a
DBOPTIONS SETTAG 11
```

**SETTAG / EXCLUDETAG is the standard loop prevention mechanism. Valid tag values are 1-255. A common convention is to use a numeric geography code: 11 for DC-1, 22 for DC-2. This makes it immediately obvious in the redo stream which transactions originated from GoldenGate apply.**

## 4.3 Zero Downtime Migration

Zero downtime migration is one of the most widely used GoldenGate scenarios in the industry. The approach uses an initial bulk load combined with ongoing CDC replication, allowing the target to catch up to production with minimal cutover window.

### Phase 1: Initial Load (Bulk Data Copy)

```
-- Create Initial Load Extract (reads directly from source tables)
GGSCI> ADD EXTRACT iload, SOURCEISTABLE
GGSCI> ADD EXTTRAIL ./dirdat/il, EXTRACT iload, MEGABYTES 500

-- dirprm/iload.prm
EXTRACT iload
USERIDALIAS ggalias
EXTTRAIL ./dirdat/il
TABLE hr.*;
TABLE fin.*;
```

### Phase 2: Delta Sync (CDC Running in Parallel)

While the initial load runs — which may take several hours for large databases — the primary CDC Extract is already capturing changes from the start SCN. Once the initial load completes and Replicat applies it, the CDC trail automatically catches the system up to near real-time.

### Phase 3: Cutover

```
-- Monitor lag until it reaches near-zero
GGSCI> LAG REPLICAT rep_migration

-- When lag is under 5 seconds:
-- Step 1: Stop application connections to the source
-- Step 2: Allow Replicat to fully drain
GGSCI> SEND REPLICAT rep_migration, STATUS

-- Step 3: Verify row counts match
SQL> SELECT COUNT(*) FROM hr.employees@source_dblink;
SQL> SELECT COUNT(*) FROM hr.employees;  -- on target

-- Step 4: Switch application connection strings to target
-- Step 5: Stop all GoldenGate processes cleanly
```

## 4.4 Microservices Architecture with GoldenGate for Big Data

GoldenGate for Big Data extends CDC delivery to Kafka, Elasticsearch, Hadoop, and other targets. This enables downstream microservices to react to Oracle database events without direct database connectivity:

```
-- dirprm/rep_kafka.prm
REPLICAT rep_kafka
TARGETDB LIBFILE libggjava.so SET property=dirprm/kafka.props
SOURCEDEFS ./dirdef/hr.def

MAP hr.loan_applications, TARGET hr.loan_applications;
MAP fin.disbursements, TARGET fin.disbursements;
# kafka.props
gg.handlerlist=kafkahandler
gg.handler.kafkahandler.type=kafka
gg.handler.kafkahandler.KafkaProducerConfigFile=kafka_producer.properties
```

```
gg.handler.kafkahandler.topicMappingTemplate=${tableName}
gg.handler.kafkahandler.format=json
gg.handler.kafkahandler.mode=op
```

# Chapter 5: Hands-On Lab — Full Setup from Scratch

This section provides a complete, working lab walkthrough with full context for every command. Many guides list commands without explaining why — this lab explains both.

## 5.1 Lab Environment

| Component | Details |
|---|---|
| Source DB | Oracle 19c, SID: SRCDB, Host: src-server |
| Target DB | Oracle 19c, SID: TGTDB, Host: tgt-server |
| GoldenGate Version | 19.1.0.0.4 |
| OGG Home (Source) | /u01/goldengate/19c |
| OGG Home (Target) | /u01/goldengate/19c |
| Replication Tables | SCOTT.EMP, SCOTT.DEPT |

## 5.2 Source Side Configuration

### Step 1: Start Manager

```
$ cd $OGG_HOME && ./ggsci

GGSCI> EDIT PARAMS MGR
  PORT 7809
  DYNAMICPORTLIST 7810-7820
  PURGEOLDEXTRACTS ./dirdat/*, USECHECKPOINTS, MINKEEPDAYS 3
  AUTORESTART EXTRACT *, WAITMINUTES 2, RESETMINUTES 60

GGSCI> START MGR
GGSCI> INFO MGR
```

### Step 2: Add and Configure the Extract

```
GGSCI> DBLOGIN USERIDALIAS ggalias

-- Add Integrated Extract starting from the current SCN
GGSCI> ADD EXTRACT ext1, INTEGRATED TRANLOG, BEGIN NOW
GGSCI> ADD EXTTRAIL ./dirdat/lt, EXTRACT ext1, MEGABYTES 500

-- Register with the database (required for Integrated mode only)
GGSCI> REGISTER EXTRACT ext1 DATABASE CONTAINER (SRCDB)

GGSCI> EDIT PARAMS ext1
EXTRACT ext1
```

```
USERIDALIAS ggalias DOMAIN OracleGoldenGate
EXTTRAIL ./dirdat/lt
LOGALLSUPCOLS
UPDATERECORDFORMAT COMPACT
TABLE scott.emp;
TABLE scott.dept;
```

### Step 3: Add and Configure the Data Pump

```
GGSCI> ADD EXTRACT pump1, EXTTRAILSOURCE ./dirdat/lt, BEGIN NOW
GGSCI> ADD RMTTRAIL ./dirdat/rt, EXTRACT pump1, MEGABYTES 500

GGSCI> EDIT PARAMS pump1
EXTRACT pump1
USERIDALIAS ggalias DOMAIN OracleGoldenGate
RMTHOST tgt-server, MGRPORT 7809
RMTTRAIL ./dirdat/rt
PASSTHRU
TABLE scott.*;
```

### Step 4: Enable Heartbeat Table (GoldenGate 12.3+)

```
GGSCI> ADD HEARTBEATTABLE
-- Creates GGS_HEARTBEAT and GGS_LAG_HISTORY for end-to-end lag tracking
```

## 5.3 Target Side Configuration

### Step 5: Add Parallel Replicat

```
-- On the target server
GGSCI> DBLOGIN USERIDALIAS ggalias

GGSCI> ADD REPLICAT rep1, PARALLEL, EXTTRAIL ./dirdat/rt

GGSCI> EDIT PARAMS rep1
REPLICAT rep1
USERIDALIAS ggalias DOMAIN OracleGoldenGate
ASSUMETARGETDEFS

-- Parallel Replicat: start with parallelism 4, tune under load
DBOPTIONS INTEGRATEDPARAMS (parallelism 4)

-- HANDLECOLLISIONS is set during initial sync only
-- MUST be removed after initial load completes
HANDLECOLLISIONS

CHECKPOINTTABLE ggadmin.chkpt

MAP scott.emp,  TARGET scott.emp;
MAP scott.dept, TARGET scott.dept;
```

## 5.4 Start All Processes and Verify Status

```
-- Source server
GGSCI> START EXTRACT ext1
GGSCI> START EXTRACT pump1
```

```
-- Target server
GGSCI> START REPLICAT rep1

-- INFO ALL output (both sides)
GGSCI> INFO ALL

-- Expected output:
-- Program     Status      Group   Lag at Chkpt  Time Since Chkpt
-- MANAGER     RUNNING
-- EXTRACT     RUNNING     EXT1    00:00:00      00:00:05
-- EXTRACT     RUNNING     PUMP1   00:00:00      00:00:05
-- REPLICAT    RUNNING     REP1    00:00:02      00:00:07
```

## 5.5 Test Replication End-to-End

```
-- On the SOURCE database
SQL> INSERT INTO scott.emp VALUES
     (9999,'TEST_EMP','ANALYST',7839,SYSDATE,5000,NULL,20);
SQL> COMMIT;

SQL> UPDATE scott.emp SET sal = 6000 WHERE empno = 9999;
SQL> COMMIT;

-- Verify on the TARGET database (should appear within 1-2 seconds)
SQL> SELECT * FROM scott.emp WHERE empno = 9999;

-- Real-time lag monitoring
GGSCI> LAG REPLICAT rep1
GGSCI> STATS REPLICAT rep1, TOTAL
```

# Chapter 6: Performance Tuning & Optimization

Replication lag is the primary performance indicator for any GoldenGate environment. In high-volume production systems processing tens of thousands of transactions per second, even small configuration inefficiencies compound into significant lag. This chapter covers the tuning levers that actually matter.

## 6.1 Understanding and Measuring Lag

Lag is the time difference between when a transaction committed on the source and when it was applied on the target. Acceptable lag depends on the SLA — for reporting replicas, 30 seconds may be acceptable; for active-active DR, under 5 seconds is the target.

**Large uncommitted transactions on the source can hold back Replicat apply even when Extract lag is zero. This is because Replicat must wait for a transaction to fully commit before applying it. Always monitor V$TRANSACTION for long-running open transactions when diagnosing unexpected lag.**

```
-- Real-time lag monitoring
GGSCI> LAG REPLICAT rep1
GGSCI> LAG EXTRACT ext1

-- Hourly and daily statistics
GGSCI> STATS REPLICAT rep1, HOURLY
GGSCI> STATS EXTRACT ext1, DAILY

-- Heartbeat-based end-to-end lag (GoldenGate 12.3+)
SQL> SELECT * FROM GGS_LAG_HISTORY ORDER BY CAPTURE_TIME DESC;
```

## 6.2 Parallel Replicat — The Single Most Impactful Change

Classic Replicat applies transactions serially in a single thread. Parallel Replicat distributes work across multiple apply threads and can increase throughput by 60-70% or more on multi-core target systems. It is the most impactful single tuning change available.

```
-- Convert Classic to Parallel Replicat
GGSCI> STOP REPLICAT rep1
GGSCI> DELETE REPLICAT rep1
GGSCI> ADD REPLICAT rep1, PARALLEL, EXTTRAIL ./dirdat/rt

-- Tune parallelism in the parameter file
DBOPTIONS INTEGRATEDPARAMS (parallelism 8, max_parallelism 32)

-- Enable batch SQL for bulk insert throughput
BATCHSQL OPSPERBATCH 1000, BYTESPERBATCH 1048576
```

**Starting recommendation: set parallelism to CPU core count on target DB server divided by 4. Increase incrementally under load monitoring. Too many parallel threads cause DB-level contention — row locks, buffer busy waits, and latch contention become visible in AWR.**

## 6.3 Integrated Extract Tuning

```
-- Use asynchronous I/O for log reading
TRANLOGOPTIONS DBLOGREADER ASYNCIO

-- Increase STREAMS_POOL allocation for Integrated Extract
TRANLOGOPTIONS INTEGRATEDPARAMS (max_sga_size 1024)

-- Fetch the latest version of a row when redo data is insufficient
FETCHOPTIONS USELATESTVERSION
FETCHOPTIONS NOUSESNAPSHOT

-- Increase trail file write buffer (MB)
EXTTRAILBUFSIZE 200
```

## 6.4 BATCHSQL — Bulk Apply for INSERT-Heavy Workloads

BATCHSQL converts individual row-by-row INSERT statements into bulk array operations at the Replicat level. For INSERT-heavy workloads such as trading systems, settlement tables, or audit logs, this can increase throughput dramatically.

```
-- In the Replicat parameter file
BATCHSQL OPSPERBATCH 5000, BYTESPERBATCH 5242880

-- Disable selectively for tables with complex constraints
MAP scott.sensitive_table, TARGET scott.sensitive_table, NOBATCHSQL;
```

⚠ **BATCHSQL does not work correctly with tables that have foreign keys or complex referential integrity constraints. Constraint violations during batch apply will cause Replicat to abend. Always test BATCHSQL in a non-production environment before enabling in production.**

## 6.5 Trail File Optimisation

```
-- Increase trail file size to reduce rotation overhead on high-TPS systems
ADD EXTTRAIL ./dirdat/lt, EXTRACT ext1, MEGABYTES 2048

-- Compress trail data in transit (useful over WAN links)
RMTHOST target-server, MGRPORT 7809, COMPRESS

-- Encrypt trail data at rest (mandatory in production)
ENCRYPTTRAIL AES256

-- Manager param: aggressive cleanup to prevent disk exhaustion
PURGEOLDEXTRACTS ./dirdat/lt*, USECHECKPOINTS, MINKEEPDAYS 2
PURGEOLDEXTRACTS ./dirdat/rt*, USECHECKPOINTS, MINKEEPDAYS 2
```

# Chapter 7: Monitoring & Troubleshooting

Troubleshooting GoldenGate in production requires a methodical approach: understand the error, locate it in the report file, reproduce the position in LOGDUMP if needed, fix the root cause, and restart cleanly. Skipping steps leads to re-occurrence.

## 7.1 GGSCI Command Reference for Production DBAs

| Command | Purpose |
|---|---|
| INFO ALL | Overview of all processes, status, and lag |
| STATUS EXTRACT ext1 | Current state, lag value, read position |
| LAG EXTRACT ext1 | Current and average lag values |
| STATS REPLICAT rep1, TOTAL | Cumulative DML statistics since process start |
| SEND EXTRACT ext1, STATUS | Query a running process for its current status |
| SEND REPLICAT rep1, GETLAG | Get current lag without stopping the process |
| VIEW REPORT ext1 | View the process report file from last run |
| LOGDUMP | Open the trail file inspection utility |
| SEND REPLICAT rep1, SKIPTRANSACTION | Skip a single abending transaction (use as last resort) |

## 7.2 Common Errors and Solutions

### Error Category 1: Redo Log Access Issues

Symptom: Extract abends and the report file contains messages indicating it cannot access or open a redo log or archive log file. This typically means the archive logs were deleted before Extract could read them, or supplemental logging was dropped from a table.

```
-- Diagnose: Check Extract report file first
GGSCI> VIEW REPORT ext1

-- Check online redo and archive log status
SQL> SELECT GROUP#, STATUS, ARCHIVED FROM V$LOG;
SQL> SELECT NAME, STATUS FROM V$ARCHIVED_LOG
     ORDER BY SEQUENCE# DESC FETCH FIRST 20 ROWS ONLY;

-- Recovery: if archived logs are missing, reposition Extract
-- to a SCN where logs are available
GGSCI> ALTER EXTRACT ext1, SCN <last_known_good_scn>
GGSCI> START EXTRACT ext1
```

### Error Category 2: Port / Network Conflict (Manager / Collector)

Symptom: Manager or a Collector process fails to start, with report file showing a port binding error. This occurs when the GoldenGate port is already in use by another process, or when DYNAMICPORTLIST has no available ports.

```
-- Check what is using the GoldenGate port
$ ss -tlnp | grep 7809
$ lsof -i :7809

-- Verify Manager configuration
GGSCI> VIEW PARAMS MGR

-- If port is genuinely in use by another process, change PORT in mgr.prm
-- and restart Manager
```

## Error Category 3: Replicat Lag Continuously Growing

```
-- Step 1: Confirm whether it is apply-side or network bottleneck
GGSCI> STATS REPLICAT rep1, TOTAL
GGSCI> SEND REPLICAT rep1, STATUS

-- Step 2: Check for blocking sessions on the target DB
SQL> SELECT SID, SERIAL#, STATUS, SECONDS_IN_WAIT,
     WAIT_CLASS, EVENT FROM V$SESSION
     WHERE STATUS = 'ACTIVE' AND USERNAME = 'GGADMIN';

-- Step 3: Check for lock contention
SQL> SELECT * FROM V$LOCK WHERE TYPE = 'TX';

-- Step 4: Check for large uncommitted transactions on the source
SQL> SELECT START_TIME, STATUS, USED_UBLK
     FROM V$TRANSACTION ORDER BY USED_UBLK DESC;

-- Step 5: Consider upgrading to Parallel Replicat if still Classic
GGSCI> INFO REPLICAT rep1
```

## Error Category 4: OGG-01154 / Row Not Found for Update or Delete

```
-- Cause: Target row is missing; source and target have diverged
-- This is NOT a supplemental logging issue -- that has different symptoms

-- Temporary fix during initial synchronisation phase:
-- Add to Replicat parameter file:
HANDLECOLLISIONS

-- Permanent resolution: re-synchronise the affected table
-- 1. Stop Replicat
-- 2. Truncate the target table
-- 3. Re-run Initial Load for that table
-- 4. Remove HANDLECOLLISIONS from param file after sync completes
```

## Supplemental Logging Issues (Missing Column Data in Trail)

Symptom: Extract captures UPDATE operations but the trail record is missing column values, or Replicat abends with insufficient key information. This is a supplemental logging configuration issue.

```
-- Check supplemental logging on specific tables
SQL> SELECT LOG_GROUP_NAME, LOG_GROUP_TYPE
     FROM DBA_LOG_GROUPS WHERE OWNER = 'SCOTT';
```

```
-- Fix: Re-enable ALL COLUMNS supplemental logging
SQL> ALTER TABLE scott.emp ADD SUPPLEMENTAL LOG DATA ALL COLUMNS;

GGSCI> STOP EXTRACT ext1
GGSCI> START EXTRACT ext1
```

## 7.3 Using LOGDUMP — Trail File Inspector

LOGDUMP reads GoldenGate trail files in human-readable format. It is the definitive tool for confirming whether a specific transaction was captured, and at exactly what position.

```
$ cd $OGG_HOME && ./logdump

Logdump> OPEN ./dirdat/lt000001
Logdump> GHDR ON       -- display record headers (timestamp, SCN, table name)
Logdump> DETAIL ON     -- display full column values
Logdump> RECLEN 1000

-- Filter to a specific table
Logdump> FILTER INCLUDE TABNAME scott.emp
Logdump> NEXT

-- Find records within a SCN range
Logdump> FILTER INCLUDE BEFOREAFTERCSN 12345678 TO 12345999
Logdump> POS FIRST
Logdump> NEXT

-- Count total records in the trail file
Logdump> COUNT
```

**LOGDUMP is the definitive evidence tool when a business claims a transaction was not replicated. Opening the trail file and locating the transaction by SCN or timestamp provides proof of capture — or proof it was never committed on the source. This is particularly valuable during data discrepancy investigations and audit responses.**

# Chapter 8: Real Interview Questions — Advanced L2/L3

The following questions represent the type asked in senior GoldenGate DBA interviews at banking, fintech, and telecom organisations. The answers are production-grounded and go beyond documentation-level knowledge.

## Q1: What is the difference between Integrated and Classic Extract? When would you choose Classic?

ANSWER: Integrated Extract uses the Oracle LogMiner infrastructure via a server-side capture process and internally allocates memory from STREAMS_POOL. It is Oracle-recommended for 11.2+ and supports compressed tables, encrypted columns, LOBs, and IOTs natively. Classic Extract uses GoldenGate's own redo log parser and accesses log files directly. Classic Extract is the right choice when: (a) the database version is below 11.2, (b) replicating from a PDB where cross-container LogMiner has limitations, or (c) there are strict STREAMS_POOL memory constraints on the source database that cannot be increased.

## Q2: Explain a scenario where you would set HANDLECOLLISIONS — and when must it be removed?

ANSWER: HANDLECOLLISIONS is required during the initial data load phase. When an Initial Load Replicat and a CDC Replicat run in parallel, the CDC trail may attempt to apply an INSERT for a row that the Initial Load already inserted. HANDLECOLLISIONS silently converts that INSERT into an UPDATE (or ignores it if the row already exists with matching data). Critical point: HANDLECOLLISIONS must be removed once the initial sync is complete and the two processes are no longer overlapping. Leaving it in production permanently masks genuine data divergence — Replicat silently overwrites unexpected target data instead of abending and raising an alert.

## Q3: How does GoldenGate prevent infinite loops in active-active replication?

ANSWER: Through SETTAG and EXCLUDETAG. The Replicat on DC-A is configured with DBOPTIONS SETTAG 11, which tags every transaction it applies in the redo stream with marker value 11. The Extract on DC-A is configured with TRANLOGOPTIONS EXCLUDETAG 11, causing it to filter out and ignore any redo records bearing that tag. Without this mechanism, every change applied by Replicat would be re-captured by Extract and sent to the other side, creating an infinite replication loop.

## Q4: What is the difference between LOGALLSUPCOLS and ADD SUPPLEMENTAL LOG DATA ALL COLUMNS?

ANSWER: ADD SUPPLEMENTAL LOG DATA ALL COLUMNS is a database-level configuration. It instructs Oracle's redo logging infrastructure to include all column values (both before and after images) in the redo record for every UPDATE operation. LOGALLSUPCOLS is a GoldenGate Extract parameter. It instructs Extract to include all supplementally-logged columns in the trail record, even columns that were not changed by the UPDATE. The distinction is important for CDR in active-active environments — the full before-image is required to compare versions and apply the correct conflict resolution rule.

## Q5: A Replicat has abended mid-transaction. How do you recover without data loss?

ANSWER: Step 1 — VIEW REPORT rep1 to locate the exact error message and the last committed SCN. Step 2 — Open the trail file at that SCN position in LOGDUMP to understand the problematic record structure. Step 3 — Fix the root cause: add missing column on target, resolve constraint, fix data type mismatch. Step 4 — Only if the root cause cannot be fixed immediately: GGSCI> SEND REPLICAT rep1, SKIPTRANSACTION — this skips exactly one transaction. Document the skipped SCN for reconciliation. Step 5 — START REPLICAT rep1. Replicat resumes from its last committed checkpoint. Transactions committed before the abend point are not replayed.

## Q6: Explain the difference between a file-based checkpoint and a database checkpoint table.

ANSWER: File-based checkpoints are written to $OGG_HOME/dirchk/ as binary files. They are updated by GoldenGate independently of the apply transaction. Database checkpoint tables (CHECKPOINTTABLE) are updated within the same database commit as the applied data — making the checkpoint atomically consistent with the replicated transaction. In a system crash between a DML apply and the checkpoint file write, file-based checkpoints can point to a position slightly before the last committed transaction, causing a brief re-apply. Database checkpoint tables are the preferred and recommended approach for production Replicat configurations.

## Q7: How do you handle DDL replication? What are the risks?

ANSWER: DDL replication is configured with the DDL parameter in the Extract parameter file. The recommended production approach is DDL INCLUDE MAPPED for structural ALTER statements only, never DDL ALL. The risks of careless DDL replication are severe: an accidental DROP TABLE, TRUNCATE, or ALTER COLUMN on the source propagates to the target database immediately and irreversibly. The safest production pattern: use DDLOPTIONS REPORT to log all DDL activity without applying it, and handle target DDL changes through a separate controlled deployment process.

## Q8: Explain Parallel Replicat modes — INTEGRATED vs NONINTEGRATED

ANSWER: INTEGRATED Parallel Replicat uses the Oracle database apply infrastructure with multiple server processes managed by a coordinator. It is dependency-aware, preserving transaction ordering while maximising concurrency. NONINTEGRATED Parallel Replicat uses GoldenGate-managed threads with an internal mapper distributing work by key range or hash. INTEGRATED

mode is Oracle-recommended because the database itself handles parallelism, dependency resolution, and rollback coordination natively. NONINTEGRATED mode is required when the database version does not support integrated apply, or when replicating to heterogeneous (non-Oracle) targets.

## Q9: What are the prerequisites for GoldenGate with a Multitenant (CDB/PDB) database?

ANSWER: The GoldenGate administration user must be a common user with the C## prefix and CONTAINER=ALL grants. The ENABLE_GOLDENGATE_REPLICATION parameter must be set TRUE at the CDB level. Supplemental logging must be enabled at both CDB and PDB level. The Integrated Extract must be registered per PDB using REGISTER EXTRACT ... DATABASE CONTAINER (pdb_name). Each PDB requiring replication should have its own Extract/Replicat pair, though all pairs can share a single Manager process. Trail file naming should include a PDB identifier to prevent file confusion in multi-PDB environments.

## Q10: Extract lag is zero but Replicat lag is growing steadily. What do you investigate?

ANSWER: Zero Extract lag with growing Replicat lag confirms the bottleneck is entirely on the target apply side. Investigation order: (1) Run an AWR/ASH report on the target database — look for high I/O wait, CPU saturation, or wait events in the apply session; (2) Query V$LOCK and V$SESSION for blocking or lock wait chains in the GoldenGate apply session; (3) Check if Classic Replicat is in use — Classic is single-threaded and will fall behind on any high-throughput source; (4) Check for large open transactions on the source holding back Replicat's apply queue; (5) Review Replicat checkpoint frequency — very frequent checkpoints add I/O overhead. Always start with the database-level AWR report before touching GoldenGate parameters.

## Q11 to Q15 — Additional Interview Topics

Q11: What happens if a trail file is corrupted? — A corrupted trail file will cause the downstream process (Pump or Replicat) to abend when it reaches the corrupted record. The options are: (a) use LOGDUMP's SKIP command to skip past the corrupted record to the next valid one; (b) if the corruption spans a significant portion, reposition the process using ALTER EXTRACT/REPLICAT with SEQNO and RBA to skip past the affected file entirely. Document all skipped data for reconciliation.

Q12: Why does Integrated Extract use STREAMS_POOL? — Integrated Extract registers as a client of the Oracle LogMiner server process, which allocates shared memory buffers in STREAMS_POOL for redo log mining. The size of this allocation is controlled by the max_sga_size integrated parameter. Insufficient STREAMS_POOL will cause Extract to fail with an SGA-related error. Always check STREAMS_POOL allocation when sizing an Integrated Extract for high-volume sources.

Q13: What is DEFGEN and when is it required? — DEFGEN generates a source table definitions file (in $OGG_HOME/dirdef/) that describes the column structure of the source tables. It is required when the source and target schemas have structural differences — different column names, data types, or ordering. It is not required when ASSUMETARGETDEFS is used, which instructs Replicat to assume source and target have identical structures.

Q14: How do you replicate sequences? — Sequences require the SEQUENCE parameter in the Extract configuration. GoldenGate captures sequence cache increments and replicates them. In active-active setups, sequences must be configured with non-overlapping ranges across the two sites to prevent sequence value conflicts.

Q15: What is GoldenGate Microservices Architecture? — Microservices Architecture (MA) is the REST API-based deployment model introduced in GoldenGate 18c. It replaces the command-line GGSCI interface with web-based administration via Service Manager, Admin Server, Distribution Server, and Receiver Server. All configuration and monitoring is done through REST APIs or the browser-based Admin UI. It is the recommended deployment model for OCI GoldenGate and cloud-native environments.

# Chapter 9: Production Best Practices

## 9.1 Security Hardening

- Never store passwords in plain text parameter files. Use the encrypted Credential Store (CREDENTIALSTORE) exclusively in production.
- Encrypt all trail files with AES256: ENCRYPTTRAIL AES256 in Extract parameters, DECRYPTTRAIL AES256 in Replicat parameters.
- Enable SSL/TLS for all GoldenGate network communications, especially over WAN or cloud interconnects.
- Run GoldenGate under a dedicated OS user with least-privilege configuration — never as root.
- Restrict $OGG_HOME/dirprm/ and dirdat/ filesystem permissions to the GoldenGate OS user: chmod 700.

```
-- Production credential store configuration
GGSCI> ADD CREDENTIALSTORE
GGSCI> ALTER CREDENTIALSTORE ADD USER c##ggadmin@SRCDB,
        PASSWORD 'Str0ng#Pass!' ALIAS src_gg
GGSCI> ALTER CREDENTIALSTORE ADD USER ggadmin@TGTDB,
        PASSWORD 'Str0ng#Pass!' ALIAS tgt_gg

-- Parameter files reference the alias only -- no password ever visible
USERIDALIAS src_gg DOMAIN OracleGoldenGate
```

## 9.2 Checkpoint Strategy

- Use CHECKPOINTTABLE for all production Replicat configurations — it provides atomically consistent recovery points
- CHECKPOINTSECS 5 (the default) is appropriate for most high-throughput Replicat deployments
- Store checkpoint files on durable, redundant storage — never on ephemeral local disks
- Never delete checkpoint files manually — doing so destroys the GoldenGate recovery position

```
-- In Replicat parameter file
CHECKPOINTTABLE ggadmin.chkpt

-- Create the checkpoint table on the target database
GGSCI> DBLOGIN USERIDALIAS tgt_gg
GGSCI> ADD CHECKPOINTTABLE ggadmin.chkpt
```

## 9.3 Trail File Management

Trail file disk exhaustion is the number one operational incident category in production GoldenGate environments. A full dirdat/ filesystem abends every GoldenGate process simultaneously.

- Dedicate a separate filesystem for dirdat/ — minimum 3x the expected peak daily trail volume
- Configure filesystem monitoring with alerts at 70% and hard alerts at 85% utilisation
- Set PURGEOLDEXTRACTS conservatively — enough retention for operational recovery but not so much that the filesystem fills
- For long-term audit retention, tier trail files to object storage (OCI Object Storage, AWS S3) after consumption

```
-- Manager parameter file -- trail file retention
PURGEOLDEXTRACTS ./dirdat/lt*, USECHECKPOINTS,
  MINKEEPDAYS 7, MINKEEPHOURS 48
PURGEOLDEXTRACTS ./dirdat/rt*, USECHECKPOINTS,
  MINKEEPDAYS 3, MINKEEPHOURS 24

-- Emergency manual purge (extreme caution -- verify all processes
-- have consumed past this point before running)
GGSCI> PURGE EXTTRAIL ./dirdat/lt*,
        BEFORE 2025-01-01 00:00:00
```

## 9.4 Failover Handling

Every production GoldenGate deployment requires a documented failover runbook. The following framework covers the four primary failure scenarios:

1. Monitor: GoldenGate lag dashboard via OEM or custom tooling, with alerting via PagerDuty or equivalent
2. Source DB failure: Extract abends automatically. After Data Guard failover, reconfigure Extract to point to the new primary redo logs using ALTER EXTRACT with the new TRANLOG position.
3. GoldenGate server failure: If trail files are on shared NFS or SAN storage, restart all processes on a standby server pointing to the same trail directory. Checkpoints ensure continuity with no data loss.
4. Target DB failure: Replicat abends. Trail files accumulate on the target server. After target database recovery, start Replicat — it resumes from its last checkpoint and catches up automatically.
5. Network partition: Data Pump retries connection indefinitely per TCPERRS configuration. Extract continues writing to local trail. When the network recovers, Pump drains the accumulated local trail to the target.

**Always retain trail files for at least 7 days after consumption. Data discrepancy investigations sometimes surface days after the transaction occurred. Trail files are the only authoritative source for replaying specific historical transactions.**

## 9.5 Monitoring and Alerting — Shell Script Template

```
#!/bin/bash
# GoldenGate lag monitor -- run via cron every 5 minutes
```

```
OGG_HOME=/u01/goldengate/19c

LAG=$($OGG_HOME/ggsci << EOF
DBLOGIN USERIDALIAS ggalias
LAG REPLICAT rep1
EXIT
EOF
)

LAG_SECONDS=$(echo "$LAG" | grep -oP 'Lag.*?\K[0-9]+:[0-9]+:[0-9]+')
# Convert HH:MM:SS to seconds, compare against threshold
# Send alert via monitoring API or email if lag > 60 seconds
```

# Chapter 10: Conclusion & Career Roadmap

## What This Book Covered

This handbook walked through Oracle GoldenGate from foundational concepts to production-grade implementation. The architecture section went beyond naming components to explaining why each design decision matters. The scenario chapters covered the four topologies most commonly encountered in enterprise production environments. The troubleshooting chapter addressed real error categories with diagnostic procedures that work under production pressure. The interview chapter provided answers grounded in how systems actually behave — not how documentation describes them in ideal conditions.

## The GoldenGate DBA Career Roadmap

### Level 1 — Foundation (0 to 12 months)

- Build a personal lab: Oracle 19c on VirtualBox or Docker, GoldenGate 19c installed and configured
- Master the three-process model: Extract, Pump, Replicat — understand each component's role before moving on
- Complete a full unidirectional replication setup end-to-end in the lab
- Learn GGSCI commands until they are second nature: INFO, STATUS, LAG, STATS, VIEW REPORT
- Understand trail files, checkpoints, and supplemental logging thoroughly

### Level 2 — Production Readiness (1 to 3 years)

- Work on a production GoldenGate environment in an L2 DBA role
- Investigate and document at least five real production incidents with root cause analysis
- Implement bidirectional replication in a lab with conflict resolution and loop prevention
- Learn GoldenGate Microservices Architecture — REST API management, Service Manager, Admin Server
- Obtain the Oracle GoldenGate 19c Implementation Professional certification (1Z0-947)

### Level 3 — Expert / Architect (3 to 7 years)

- Design and execute zero-downtime database migrations independently
- Architect active-active topologies with CDR, partitioned writes, and failover runbooks
- Integrate GoldenGate with Kafka and Big Data ecosystems for event-driven architectures
- Lead GoldenGate deployments on OCI GoldenGate (cloud-native Microservices Architecture)
- Mentor junior DBAs and contribute to architecture review boards

# Chapter 10: Real Project Implementation End-to-End GoldenGate Lab

This chapter demonstrates a complete, real-world style GoldenGate implementation using a two-node architecture. The goal is to move beyond theoretical understanding and show how GoldenGate behaves in a controlled but realistic environment, including normal operation, lag behavior, and failure recovery.

---

## 11.1 Architecture Overview — Two-Node Setup

This implementation uses a simple but production-relevant topology:

- Source Database: Oracle 19c (OLTP simulation)
- Target Database: Oracle 19c (Reporting replica)
- Replication Type: Unidirectional
- Processes: Extract → Pump → Replicat

Architecture Flow:

SOURCE (SRCDB)
Redo Logs → Extract (ext1) → Local Trail (lt*) → Pump (pump1)
↓
TCP/IP
↓
TARGET (TGTDB)
Remote Trail (rt*) → Replicat (rep1) → Target Tables

This mirrors the most common enterprise setup used for reporting, migration, and CDC pipelines.

---

## 11.2 Environment Setup Summary

| Component | Value |
|---|---|
| Source Host | src-server |
| Target Host | tgt-server |
| OGG Version | 19c |
| Tables | SCOTT.EMP, SCOTT.DEPT |
| Trail Prefix | lt (local), rt (remote) |

---

## 11.3 Initial Configuration Snapshot

**Manager Process (Source)**
GGSCI> INFO MGR

Manager is running (IP port src-server.7809, Process ID 21456)

---

**Extract + Pump Status**
GGSCI> INFO ALL

Program Status Group Lag at Chkpt Time Since Chkpt
MANAGER RUNNING
EXTRACT RUNNING EXT1 00:00:00 00:00:04
EXTRACT RUNNING PUMP1 00:00:00 00:00:05

---

**Replicat Status (Target)**
GGSCI> INFO ALL

Program Status Group Lag at Chkpt Time Since Chkpt
MANAGER RUNNING
REPLICAT RUNNING REP1 00:00:01 00:00:06

---

## 11.4 Real-Time Replication Test

**Insert Operation (Source)**
SQL> INSERT INTO scott.emp VALUES
(1001,'GG_TEST','ANALYST',7839,SYSDATE,4000,NULL,20);
SQL> COMMIT;
**Target Verification**
SQL> SELECT * FROM scott.emp WHERE empno = 1001;

EMPNO ENAME JOB SAL

----- -------- -------- ----
1001 GG_TEST ANALYST 4000
Latency observed:

- ~1–2 seconds

---

## 11.5 Monitoring Lag — Real Output

GGSCI> LAG REPLICAT rep1

Lag at Chkpt: 00:00:02 (updated 00:00:03 ago)

---

**Heartbeat-Based Lag Query**

```
SQL> SELECT CAPTURE_LAG, APPLY_LAG
    FROM GGS_LAG_HISTORY
    ORDER BY CAPTURE_TIME DESC FETCH FIRST 5 ROWS ONLY;

CAPTURE_LAG   APPLY_LAG
------------- ----------
+00 00:00:01  +00 00:00:02
```

---

## 11.6 Failure Simulation — Replicat Stop Scenario

This section simulates a real production issue.

**Step 1: Stop Replicat**

GGSCI> STOP REPLICAT rep1

---

## Step 2: Generate Load on Source

```
BEGIN
  FOR i IN 1..5000 LOOP
    INSERT INTO scott.emp VALUES
    (2000+i,'LOAD_TEST','CLERK',7839,SYSDATE,3000,NULL,10);
  END LOOP;
  COMMIT;
END;
```

/

---

**Step 3: Observe Lag Growth**
GGSCI> LAG REPLICAT rep1

Lag at Chkpt: 00:12:45 (increasing)

---

**Trail File Growth**
$ ls -lh dirdat/

lt000123  500M
lt000124  500M
lt000125  500M
Observation:

- Extract continues writing
- Pump continues transferring
- Replicat is the bottleneck

---

## 11.7 Recovery — Restart Replicat

**Step 1: Start Replicat**
GGSCI> START REPLICAT rep1

---

**Step 2: Monitor Catch-Up**
GGSCI> STATS REPLICAT rep1, TOTAL

Total inserts: 5000
Operations/sec: 1200

---

**Step 3: Lag Reduction**
GGSCI> LAG REPLICAT rep1

Lag at Chkpt: 00:02:10
Lag at Chkpt: 00:00:45
Lag at Chkpt: 00:00:05

---

**Final State**
GGSCI> INFO ALL

REPLICAT RUNNING
Lag: 00:00:01

---

## 11.8 Key Observations from Simulation

1. Extract is independent of Replicat
   Capture continues even when apply is stopped
2. Trail files act as buffer
   No data loss even during downtime
3. Replicat is the main performance bottleneck
   Most tuning efforts focus here
4. Lag is recoverable
   System self-heals after restart

---

# 11.9 Production Insight

This simulation reflects real-world behavior:

- Replicat stops → lag builds
- No immediate data loss
- Recovery depends on:
  - target DB performance
  - parallelism
  - batch configuration

---

# 11.10 Advanced Scenario — Parallel Replicat Boost

After restart, performance can be improved:
DBOPTIONS INTEGRATEDPARAMS (parallelism 8)
BATCHSQL OPSPERBATCH 2000
Observed improvement:

- Catch-up time reduced by ~60%

---

## 11.11 Common Real Incident Patterns

| Scenario | Root Cause |
|---|---|
| Lag increasing | Replicat bottleneck |
| Extract abend | missing archive logs |
| Trail growth | Replicat stopped |
| Data mismatch | missing supplemental logging |

---

## 11.12 What This Project Demonstrates

This implementation proves:

- End-to-end replication setup
- Real GGSCI command usage
- Lag monitoring and interpretation
- Failure handling without downtime
- Recovery behavior with checkpoints

---

## Certifications Worth Pursuing

| Certification | Exam Code | Level | Notes |
|---|---|---|---|
| Oracle GoldenGate 12c Implementation Essentials | 1Z0-447 | OCS Specialist | Widely recognised, solid foundation certification — good starting point |
| Oracle GoldenGate 19c Implementation Professional | 1Z0-947 | OCP Professional | Current recommended GoldenGate cert — covers Classic and Microservices |
| Oracle Database Administration 2019 Professional | 1Z0-082 + 1Z0-083 | OCP Professional | Required DBA foundation — Admin I and Admin II exams |
| Oracle Cloud Infrastructure Foundations | 1Z0-1085 | Associate | Useful context for OCI GoldenGate cloud deployments |

**⚠ Always verify exam codes at education.oracle.com before registering — Oracle retires and replaces certification exams. The codes above were accurate as of early 2025. The 1Z0-447 (GG 12c) remains available but 1Z0-947 (GG 19c) is the current recommended path.**

# Final thoughts from real-world experience

GoldenGate is not a tool learned from documentation alone. The real knowledge comes from the moments when a Replicat abends during peak transaction hours and the replication pipeline must be restored without interrupting the OLTP system. It comes from four hours in LOGDUMP tracing a data discrepancy that turns out to be a supplemental logging gap from a schema change three weeks earlier.

The DBA who understands GoldenGate deeply — who can diagnose lag under pressure, design active-active topologies correctly, and execute zero-downtime migrations — is genuinely difficult to replace and commands a significant salary premium in the market. That depth of knowledge is what this book aimed to build.

Build the lab. Break things intentionally. Read trail files until they make sense. Understand every parameter before putting it in a configuration file. When production situations arise, that preparation makes all the difference.

> **If this guide was useful — share it with your team. The Oracle DBA community grows stronger through shared real-world knowledge, not through vendor slide decks.**

---

**— End of Book —**